

Examen Segunda Convocatoria, Franja Horaria Mañana

```
# Your imports HERE !!!!
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

01 - 8pts

Cargar el dataset adjunto de datos de vuelos 'flight_data.csv'

- A - Filtrar los vuelos con retrasos superiores a 30 minutos
- B - Escribir una función que clasifique los vuelos como 'A tiempo' (menos de 15 minutos), 'Retrasados' (menos de 60 minutos) o 'Cancelados' (más de 60 minutos), úsala para crear una nueva columna en el dataframe llamada 'Status'
- C - Visualizar el número total de vuelos por aerolínea, ordenados de forma descendente
- D - Visualizar la distribución de los retrasos de los vuelos mediante un histograma
- E - Filtrar los vuelos procedentes de 'JFK' y 'LAX', calcula la media y la mediana de los retrasos por aerolínea
- F - Dibujar un gráfico de barras con la media del retraso por aerolínea de 'JFK' y 'LAX' del punto anterior
- G - Mostrar las 3 compañías aéreas con la mediana de retrasos de vuelo más alta
- H - Filtrar los vuelos que hayan tenido un retraso por encima de la media de al menos la desviación estándar, y calcular el retraso medio de ellos

```
# Your solution HERE !!!!
df = pd.read_csv('flight_data.csv')
df.head(5)
```

	Flight_Number	Origin	Destination	Airline	Delay_Minutes
0	FL8270	ATL	HND	American	147
1	FL1860	LAX	HND	United	163
2	FL6390	LAX	LHR	United	146
3	FL6191	ATL	HND	American	89
4	FL6734	DFW	SFO	British Airways	146

```
# A - Filtrar los vuelos con retrasos superiores a 30 minutos
df_filtrado = df[df['Delay_Minutes']>30]
```

```
# B - Escribir una función que clasifique los vuelos como 'A tiempo'
(menos de 15 minutos), 'Retrasados' (menos de 60 minutos) o
'Cancelados' (más de 60 minutos),
# úsala para crear una nueva columna en el dataframe llamada 'Status'
def clasificar_vuelo(delay):
```

```

    if delay < 15:
        return 'A tiempo'
    if delay < 60:
        return 'Retrasados'
    if delay >=60:
        return 'Cancelados'

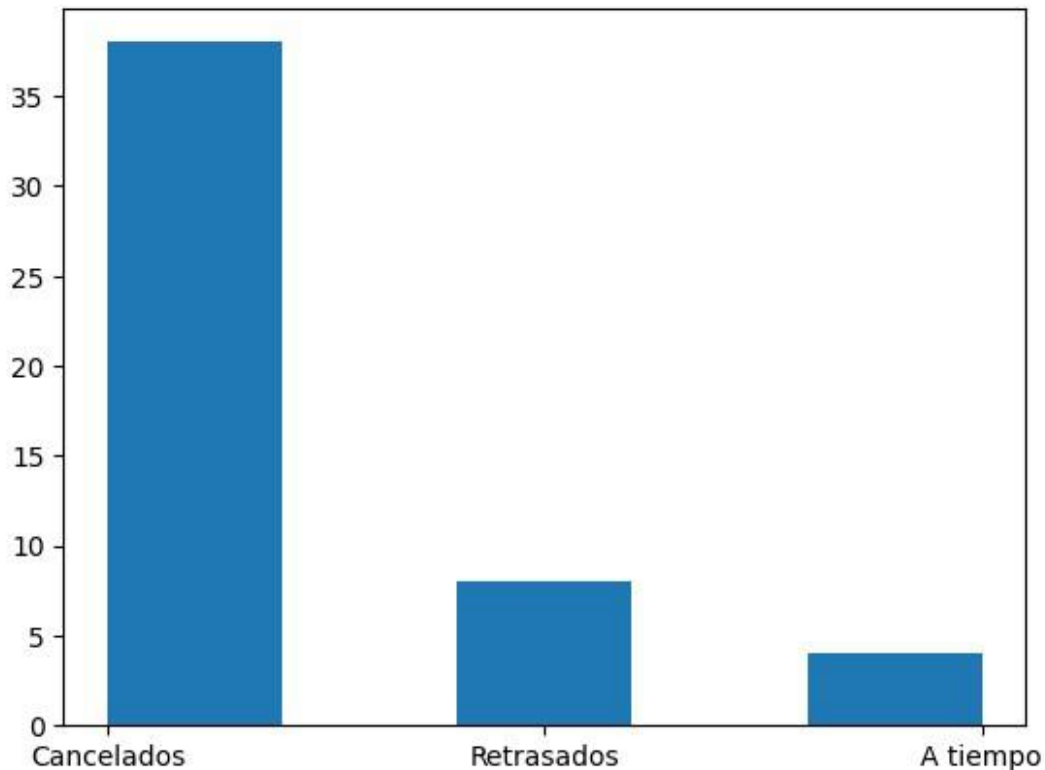
df['Status'] = df['Delay_Minutes'].apply(clasiificar_vuelo)

# C - Visualizar el número total de vuelos por aerolínea, ordenados de
forma descendente
vuelos_aerolineas = df['Airline'].value_counts(ascending=False)
vuelos_aerolineas

Airline
American      15
United        12
Southwest      9
Delta          8
British Airways 6
Name: count, dtype: int64

# D - Visualizar la distribución de los retrasos de los vuelos
mediante un histograma
fig, ax = plt.subplots()
ax.hist(df['Status'], bins= 5)
plt.show()

```



E - Filtrar los vuelos procedentes de 'JFK' y 'LAX', calcula la media y la mediana de los retrasos por aerolínea

```
df_filtrado = df[df['Origin'].isin(['JFK', 'LAX'])]
mean = df_filtrado.groupby('Airline')['Delay_Minutes'].mean()
median = df_filtrado.groupby('Airline')['Delay_Minutes'].median()
display(mean)
display(median)
```

```
Airline
American      100.000000
British Airways  63.000000
Delta         131.000000
Southwest     74.833333
United        138.428571
Name: Delay_Minutes, dtype: float64
```

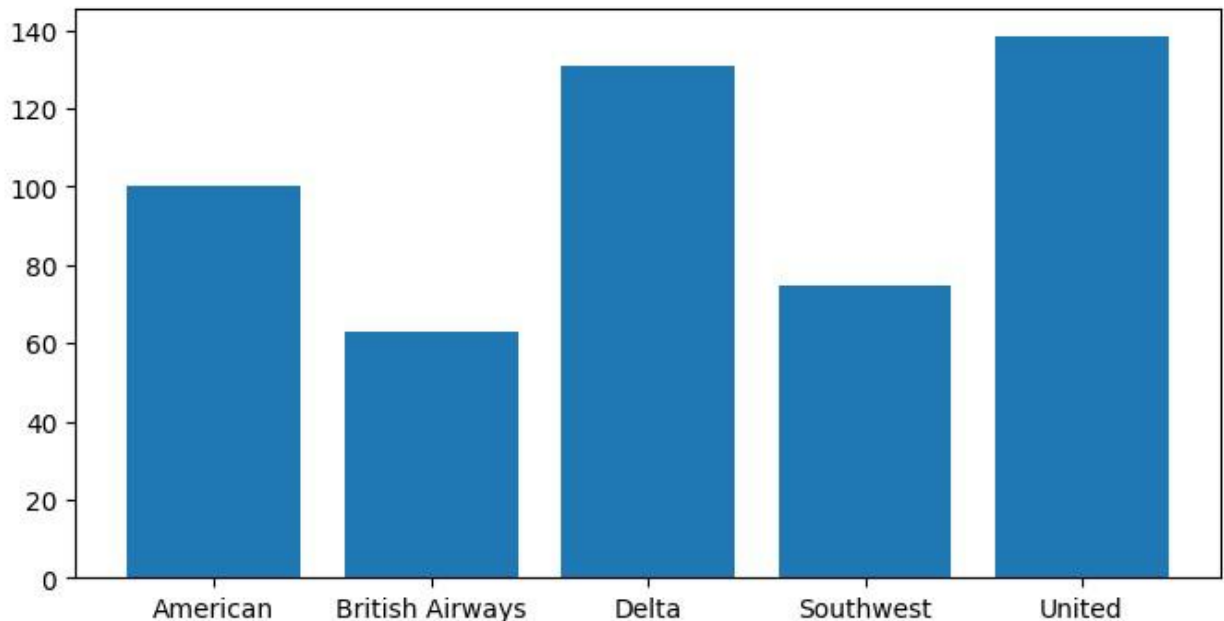
```
Airline
American      81.0
British Airways  53.0
Delta         131.0
Southwest     79.5
United        151.0
Name: Delay_Minutes, dtype: float64
```

F - Dibujar un gráfico de barras con la media del retraso por aerolínea de 'JFK' y 'LAX' del punto anterior

```
fig, ax = plt.subplots(figsize=(8, 4))

ax.bar(mean.index, mean.values)

plt.show()
```



G - Mostrar las 3 compañías aéreas con la mediana de retrasos de vuelo más alta

```
max_median = median.nlargest(3)
print(max_median)
```

```
Airline
United      151.0
Delta       131.0
American     81.0
Name: Delay_Minutes, dtype: float64
```

#H - Filtrar los vuelos que hayan tenido un retraso por encima de la media de al menos la desviación estándar, y calcular el retraso medio de ellos

```
df['mean_airline'] = df.groupby('Airline')
['Delay_Minutes'].transform('mean')
df['mean_airline'] = df.groupby('Airline')
['Delay_Minutes'].transform('mean')
df['std_airline'] = df.groupby('Airline')
['Delay_Minutes'].transform('std')
```

```
vuelos_filtrados = df[df['Delay_Minutes'] > df['mean_airline'] +
df['std_airline']]
```

```
mean_filtrados = vuelos_filtrados.groupby('Flight_Number')
```

```
['Delay_Minutes'].mean().reset_index(name='Retraso medio')
display(mean_filtrados)
```

	Flight_Number	Retraso medio
0	FL4152	165.0
1	FL6578	160.0
2	FL7265	147.0
3	FL7420	177.0
4	FL8270	147.0
5	FL9433	151.0

02 - 2pt

- Escribir una función 'prime_number' que cada vez que se invoque devuelva el siguiente número primo, empezando por el 2, hasta cualquier número infinito. Usando plain Python, sin ninguna librería. Primo es aquel número natural mayor que 1, que solo es divisible por 1 y por si mismo.

```
# Your solution HERE !!!!
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def prime_number():
    n = 2
    while True:
        if is_prime(n):
            yield n
        n += 1

# Test Use Case
expected_output = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

g = prime_number() # This calls your function named prime_number
output = [next(g) for n in range(len(expected_output))]

display(expected_output)
display(output)

assert(expected_output == output) # This will fail if the output is
not as expected

[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```